```
;               Write X86/64 ALP to perform overlapped block
transfer with string specific instructions
;               Block containing data can be defined in the data
segment.

; Macro to write to stdout
%macro write 2
        mov rax, 1      ; syscall number for sys_write
        mov rdi, 1      ; file descriptor 1 (stdout)
        mov rsi, %1     ; buffer address
        mov rdx, %2     ; number of bytes to write
        syscall
%endmacro

; Macro to read from stdin
%macro read 2
        mov rax, 0      ; syscall number for sys_read
        mov rdi, 0      ; file descriptor 0 (stdin)
        mov rsi, %1     ; buffer address
        mov rdx, %2     ; number of bytes to read
        syscall
%endmacro

section .data
        ; Welcome message and information
        dispMsg db "Write X86/64 ALP to perform
overlapped block transfer with string specific instructions
Block containing data can be defined in the data
segment.",10, "Name:- Sumaanyu",10,"Roll:- 7256",10,"Date
of Performance:- 24/03/2025",10
        dispMsgLen equ $-dispMsg

        ; Menu display
        msg db "------------------MENU------------------", 10
          db "      1. Without string instruction ", 10
          db "      2. With string instruction ", 10
          db "      3. Exit ", 10
          db "Enter your choice : "
        msglen equ $-msg

        ; Prompt label
        msg1 db "Number : "
        len equ $-msg1

        endl db 10

        ; Source and destination arrays
        src dq 10h, 20h, 30h, 40h, 50h      ; Source block
        dest dq 0h, 0h, 0h, 0h, 0h        ; Destination block

        tab db 20h                  ; Tab character
        b db "Before : ", 10          ; Before transfer
        a db "After  : ", 10          ; After transfer
        l equ $-a

section .bss
        temp resb 16                ; Temp buffer for hex
display
        choice resb 2               ; User input for menu
choice

section .text
        global _start
_start :
        write dispMsg, dispMsgLen       ; Display initial
info

main_menu :
        write msg, msglen           ; Show menu
        read choice, 2              ; Read user's choice

        ; Compare input with '1', '2', '3'
        cmp byte[choice], "1"
        je case1

        cmp byte[choice], "2"
        je case2

        cmp byte[choice], "3"
        je case3

; ============ Case 1: Without string instructions
============
case1 :
        write b, l                  ; Display "Before :"

        ; Zero-out destination
        mov rcx, 5
        mov rsi, dest
        mov rax, 0
init1 :
        mov [rsi], rax
        add rsi, 8
        loop init1

        call displayBlock           ; Display blocks
        write endl, 1

        ; Overlapped block transfer (manual method)
        write a, l                  ; Display "After :"
        mov rcx, 6                  ; 6 elements to shift
        mov rsi, src
        add rsi, 40                 ; rsi -> end of source
        mov rdi, dest
        add rdi , 24                ; rdi -> middle of destination

next :
        mov rax, [rsi]
        mov [rdi], rax
        sub rsi, 8
        sub rdi, 8
        loop next

        ; Clear some of source for overlap effect
        mov rax, 0
        mov rsi, src
        mov rcx, 3
next3 :
        mov [rsi], rax
        add rsi, 8
        loop next3
```

```
        call displayBlock
        write endl, 1
        jmp main_menu

; ============ Case 2: With string instructions
============
case2 :
        write b, l              ; Display "Before :"

        ; Zero-out destination
        mov rcx, 5
        mov rsi, dest
        mov rax, 0
init2 :
        mov [rsi], rax
        add rsi, 8
        loop init2

        call displayBlock
        write endl, 1

        ; Overlapped block transfer using string
instruction (MOVSQ)
        write a, l
        mov rcx, 6
        mov rsi, src
        add rsi, 40             ; rsi -> last element of
source
        mov rdi, dest
        add rdi, 24             ; rdi -> middle of destination
        std                     ; Set direction flag (right to left)
next2 :
        movsq                   ; Move qword from [rsi] to
[rdi]
        loop next2

        ; Clear part of the source block
        mov rax, 0
        mov rsi, src
        mov rcx, 3
next4 :
        mov [rsi], rax
        add rsi, 8
        loop next4

        call displayBlock
        write endl, 1
        jmp main_menu

; ============ Case 3: Exit ============
case3 :
        mov rax, 60             ; syscall: exit
        mov rdi, 0
        syscall

; ============ Display Block Contents ============
displayBlock :
        mov rcx, 5
        mov rsi, src            ; Point to source array

nextD :
        push rcx
        push rsi
        mov rbx, [rsi]          ; Load value at [rsi]
        call display            ; Show hex value
        write tab, 1
        pop rax
        mov rbx, rax
        push rax
        call display            ; Show address
        write endl, 1
        pop rsi
        pop rcx
        add rsi, 8
        loop nextD

        write endl, 1
        mov rcx, 5
        mov rdi, dest           ; Point to destination array
nextA :
        push rcx
        push rdi
        mov rbx, [rdi]
        call display
        write tab, 1
        pop rax
        mov rbx, rax
        push rax
        call display
        write endl, 1
        pop rdi
        pop rcx
        add rdi, 8
        loop nextA
        ret

; ============ Display number in hex ============
display :
        mov rsi, temp
        mov rcx, 16             ; 16 hex digits (64-bit)
next1 :
        rol rbx, 4              ; Rotate left by 4 bits
        mov al, bl
        and al,
```